VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Winter Semester 2022-2023

Continuous Assessment Test – 1 KEY

SLOT A1

Programme Name &Branch: B. Tech (BCI, BCE, BCT, BDS, BCB)

Course Name & code: Data Structures and Algorithms – BCSE202L

Class Number (s): VL2022230505842, 6331, 5847, 5851, 5837, 6304, 5855, 5849, 5840
Faculty Name (s): Joshva Devadas T, Priyanka N, Sayan Sikdar, Naveenkumar J, Akash Sinha, Kalaivani K, Ganesh Shamrao Khekare, Sunil Kumar PV, S M Farooq.

Exam Duration: 90 Min. **Maximum Marks: 50**

All questions carry equal marks.

Q.No.	Question	Max Marks
1.	a) Find the time complexity for the given recurrence relation using Master's Theorem. i) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$ ii) $T(n) = 2T\left(\frac{n}{2}\right) + n\log n$ Ans: Writing master's theorem • It is used to analyse recurrence relation. • $T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k log^p n)$, $a \geq 1, b > 1, k \geq 0$ and p is real number • Case 1: $if\ a > b^k, then\ T(n) = \theta(n^{log_b a})$ • Case 2: $if\ a = b^k, then$ • a) $if\ p > -1, then\ T(n) = \theta(n^{log_b a} log^{p+1} n)$ • b) $if\ p = -1, then\ T(n) = \theta(n^{log_b a} log^2 n)$ • c) $if\ p < -1, then\ T(n) = \theta(n^{log_b a})$ Case 3: $if\ a < b^k, then$ • a) $if\ p \geq 0, then\ T(n) = \theta(n^k log^p n)$ • b) $if\ p < 0, then\ T(n) = \theta(n^k)$ i)a=3, b=2, k=2, p=0 ($a \geq 1$, $b > 1$, $k \geq 0$, p is a real number) a b^k 3 2^2 $3 < 4 \Rightarrow a < b^k$, Falls under case 3a. T(n) = $\theta(n^k log^p n)$ = $\theta(n^2 log^0 n)$ = $\theta(n^2)$ ii) a=2, b=2, k=1, p=1 a b^k 2 2^1 $2 = 2 \Rightarrow a = b^k$ Falls under case 2a. T(n) = $\theta(n^{log_b a} log^{p+1} n)$ = $\theta(n^{log_2 2} log^{1+1} n)$ = $\theta(n^1 log^2 n)$ = $\theta(n\ log^2 n)$ = $\theta(n\ log \log n)$	2 M 2 M 2 M

b) Find time complexity for the following recursive relation using back substitution method. [For this question consider the following question for 4 M]

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + 1, \ for\ n > 1 \\ 1, for\ n = 1 \end{cases}$$

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Substitute n/2 in place of n in the given recurrence relation

$$= \left[T\left(\frac{\frac{n}{2}}{2}\right) + 1\right] + 1$$

$$= T\left(\frac{n}{4}\right) + 2$$

$$= T\left(\frac{n}{2^2}\right) + 2$$

Substitute n/4 in place of n in the given recurrence relation

$$= \left[T\left(\frac{n}{8}\right) + 1\right] + 2$$

$$= T\left(\frac{n}{8}\right) + 3$$

$$= T\left(\frac{n}{2^3}\right) + 3$$

. .

$= T\left(\frac{n}{2^k}\right) + k$, [k^{th} term]

When $\frac{n}{2^k}$ reaches to 1 then it will be a base case.

$$\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$$

Substitute k value in k^{th} term.

$$= T(1) + \log_2 n$$

$$= 1 + \log_2 n$$

Applying O notation to find time complexity,

$$= O(1 + \log_2 n) = O(\log_2 n)$$

4 M

2. a) Find the time complexity for the following code (step count method)

i)	*ii)*

sum1=0
for(k=1; k<=n;k=k*2)
    for(j=1; j<=n; j++)
        sum1++
	sum=0;
	for(j=1; j<=n; j++)
	    for(i=1; i<=j; i++)
	        sum++;
	    for(k=0; k<n; k++)
	        a[k]= k;

Ans:

i)

Code	Count

sum1=0
for(k=1; k<=n;k=k*2)
    for(j=1; j<=n; j++)
        sum1++
	1
	log n
	n
	n

Total	1 + (log n) * (n+n) 1+ 2 n * log n
Applying Big O	O(n log n)

2 M

ii)

For each j value i value is repeating for j times. 3 M

```
for( j=1; j<=n; j++)
   for( i=1; i<=j; i++)
               sum++;
```

i	1	2	3	4	--	n
J	1	2 times	3 times	4 times	--	n times
sum++	1*1	1*2	1*3	1*4	--	1*n times

Finally, 1 + 2 + 3 + 4 + 5 + … n = n(n+1)/2

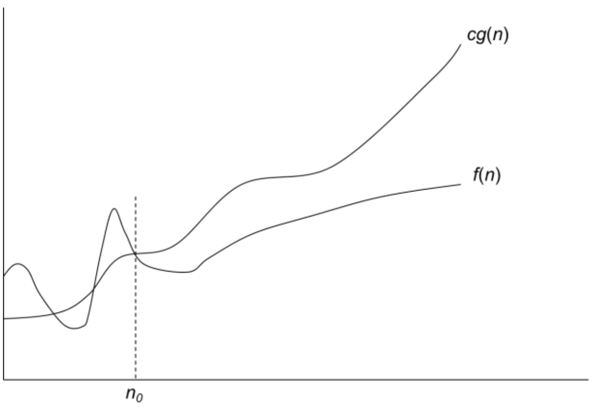
Given Code	Count

sum=0;
for(j=1; j<=n; j++)
   for(i=1; i<=j; i++)
            sum++;
   for(k=0; k<n; k++)
      a[k]= k;
	1
	n(n+1)/2
	
	n
	n

Total	$1+\frac{n*(n+1)}{2}+n+n$ $1+\frac{n^2}{2}+\frac{n}{2}+2n$
Applying Big O	$O(1+\frac{n^2}{2}+\frac{n}{2}+2n) = O(n^2)$

b) Find c and n_0 values if $f(n) = O(g(n))$ for the functions $f(n) = 3n + 2$ and $g(n) = n$.

O notation: 2 M



$f(n)=O(g(n))$: there exist positive constants c and n_0 such that

$$0 \le f(n) \le cg(n) \text{ for all } n \ge n_0$$

If $f(n) = O(g(n))$, then $f(n) \le c * g(n)$, for c > 0, $n_0 \ge 1$ 3 M

$3n+2 \le c*n$

For c=4, the above equation holds.

$3n+2 \le 4n \rightarrow 2 \le n \rightarrow n \ge 2$.

3. Convert the given Infix notation into its equivalent postfix notation and show step trace with the conversion algorithm.

Infix notation: (a/(b-c+d))*(e-a)*c

Ans:

Algorithm for conversion of Infix to postfix notation.

Let, X is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression Y.

1. Print operands as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the Incoming operator onto the stack.
3. If the Incoming symbol is '(',push it onto stack.
4. If the Incoming symbol is ')', pop the stack & add operators to output Y until left parenthesis is found.
5. If incoming symbol has higher precedence than the top of the stack, push it onto stack.
6. If the Incoming symbol has lower precedence than the top of the stack, then pop and add to the output. Then test the incoming operator against the new top of the stack.
7. If the incoming operator has equal precedence with the top of the stack, use associativity rule.
8. At the end of the expression pop all the operators of stack.

Associativity L to R then pop & print the top of the stack & then push in the incoming operator.

R to L then push the incoming operator

3 M

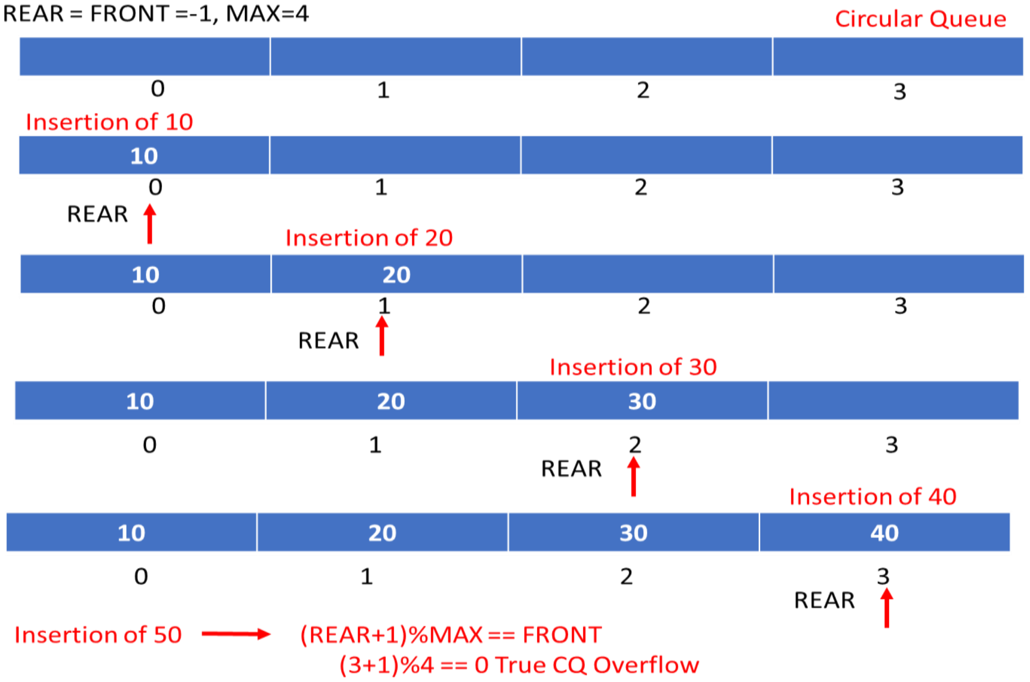
Input Infix notation: (a/(b-c+d))*(e-a)*c

S. No.	Input Symbol	Rule Number	Stack	Output
1	(	3	(	Empty
2	a	1	(	a
3	/	2	(/	a
4	(	3	(/ (	a
5	b	1	(/ (	a b
6	-	2	(/ (-	a b
7	c	1	(/ (-	a b c
8	+	7 & Associativity rule	(/ (+	a b c -
9	d	1	(/ (+	a b c – d
10	)	4	(/	a b c – d +
11	)	4	Empty	a b c – d + /
12	*	5	*	a b c – d + /
13	(	3	* (	a b c – d + /
14	e	1	* (	a b c – d + / e
15	-	2	* (-	a b c – d + / e
16	a	1	* (-	a b c – d + / e a
17	)	4	*	a b c – d + / e a -
18	*	7 & Associativity rule	*	a b c – d + / e a - *
19	c	1	*	a b c – d + / e a - * c
20		8	Empty	a b c – d + / e a - * c *

7 M

4.	a) List out the advantages of circular queue. (Any three enough)
	Ans: Page replacement algorithms
	CPU Scheduling
	Inter-process communication
	Resource allocation in operating systems
	b) Engrave pseudocode to perform operations on circular Queue. Assume the initial state of the Circular Queue is Empty with size N=4 and demonstrate the following operations with an appropriate diagram with the status of REAR and FRONT pointers.
	Instructions in sequence:
	1. Successive Insertion of the elements 10, 20, 30, 40, 50.
	2. Successive Deletion of two elements.
	3. Insertion of the element 50.
	4. Successive deletion of three elements.
	Ans:
	Let us assume an array CQ of size MAX with initial FRONT values REAR are -1.
	Algorithm CQ_Insert(ITEM)
	Step 1: IF (REAR+1)%MAX = FRONT THEN //If REAR reaches to FRONT
	Write "circular queue overflow"
	Exit
	[END OF IF]
	Step 2: IF REAR = -1 and FRONT = -1 THEN // If Circular Queue is Empty.
	REAR = FRONT=0
	ELSE
	REAR = (REAR+1)%MAX // Increment REAR value for Insert
	[END OF IF]
	Step 3: CQ[REAR]=ITEM //Insertion of ITEM into CQ
	Step 4: Exit
	
	Algorithm CQ_Delete( )
	Step 1: IF FRONT=-1 and REAR=-1 THEN // If CQ is empty
	Write "circular queue under flow"
	Exit
	[END OF IF]
	Step 2: SET VAL = CQ[FRONT] // Deleted element in VAL
	Step 3: IF FRONT = REAR THEN // If CQ has a single element
	REAR = FRONT = -1
	ELSE
	FRONT = (FRONT+1)%MAX
	[END OF IF]
	Step 4: Exit
	1. Successive Insertion of 10, 20, 30, 40, 50
		3 M
		
		3 M

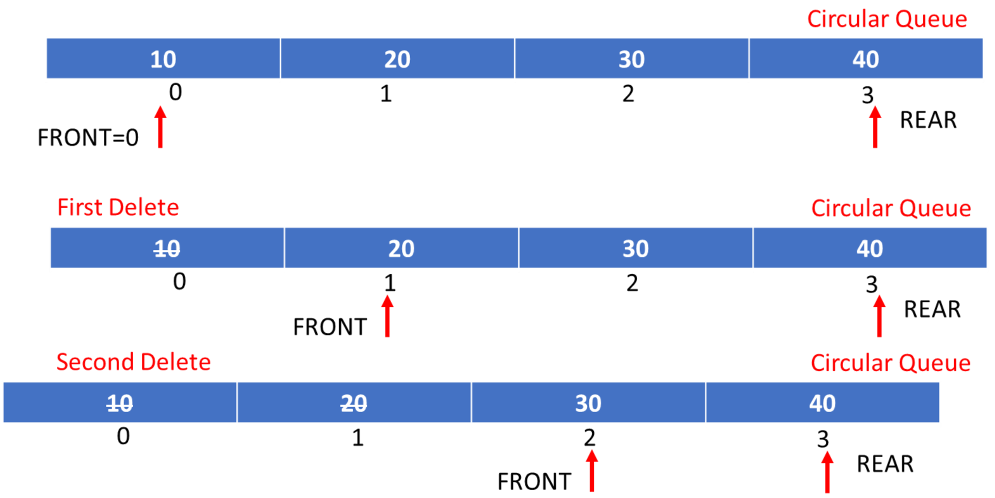
Successive Insertion of 10, 20, 30, 40, 50
REAR = FRONT =-1, MAX=4



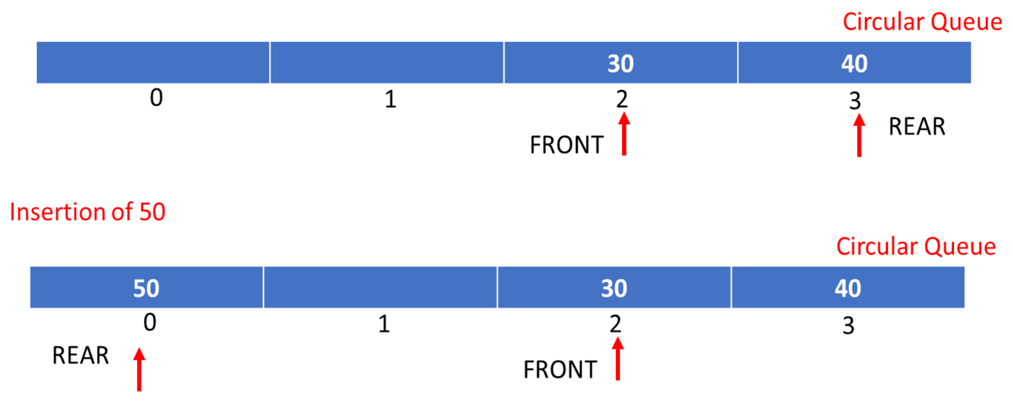
4 M

2. Successive deletion of two elements

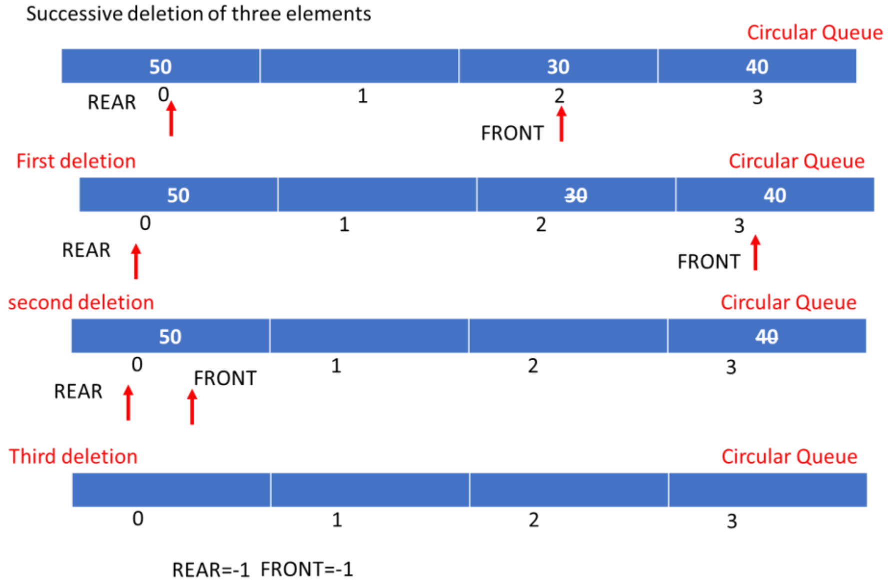
Successive Deletion of two elements



3. Insertion of the element 50.



4. Successive deletion of three elements



5. Analyze Insertion sort algorithm. Sort and give the trace for the following elements using Insertion sort. 3 M

C, A, Z, S, P, S
67, 65, 90, 83, 80, 83 (ASCII Values)

```
Insertion_Sort(A,N)
step 1: Repeat step 2 to 5 for k: = 1 to N-1
step 2:         set TEMP:= A[k]
step 3:         set J:= K - 1
step 4:         Repeat while TEMP <= A[J]
                        set A[J+1]:= A[J]
                        set J:= J - 1
                  [End of Inner While loop]
step 5:         set A[J+1] = TEMP
```

Let us an Array A of N elements is in memory. Insertion sort is one of the simplest sorting algorithms. It consists of N-1 passes.

Original	**67**	**65**	**90**	**83**	**80**	**83**	**Positions moved**
pass = 1	**65**	**67**	**90**	**83**	**80**	**83**	1
Pass = 2	**65**	**67**	**90**	**83**	**80**	**83**	0

Pass = 3	**65**	**67**	**83**	**90**	**80**	**83**	1
Pass = 4	65	67	80	83	90	83	2
Pass = 5	65	67	80	83	83	90	2

4 M

Sorted elements: A, C, P, S, S, Z
Time complexity: For each iteration (pass) no. of comparisons are
1 , 2, 3, 4, ----n-1
Adding those comparisons (1+2+3+ --- n-1) = n(n-1)/2= $O(n^2)$
Best case time complexity: $\Omega(n)$ (If all the elements are presorted)
Average case time complexity $\theta(n^2)$.

3 M

End of key